4.3.4 用综合属性代替继承属性

从先前的例子已经看出，*S* 属性定义比 *L* 属性定义易于理解。从例 4.10 还知道，改变文法可能会引入继承属性。同样，改变文法有时可以避免使用继承属性，后者对于构造只允许综合属性的翻译方案极其有用。

例如，Pascal 语言的变量声明由标识符表加类型组成，类型出现在变量表的后面，如 m，n：integer。这种声明可以用下面的文法描述：

$D \rightarrow L : T$

$T \rightarrow$ **integer** | **char**

$L \rightarrow L$, **id** | **id**

在这个文法中，标识符表由 *L* 产生，但类型不是 *L* 的子树，因此不能仅靠使用综合属性把类型和标识符联系起来。事实上，在第一个产生式中，由于非终结符 *L* 从它右边的 *T* 获得类型信息，所写的语法制导定义将不可能是 *L* 属性定义的，所以基于它的翻译也不可能在分析期间完成。

这个问题可以通过重新构造文法来解决，把类型作为标识符表的最后一个成分：

$D \rightarrow$ **id** L

$L \rightarrow$, **id** L | : T

$T \rightarrow$ **integer** | **char**

现在，类型作为 *L* 的综合属性 *L*. *type*，当标识符由 *L* 产生时，它的类型可以填入符号表，具体的翻译方案如下：

$D \rightarrow$ **id** L { *addType* (**id**. *entry*, *L*. *type*); }

$L \rightarrow$, **id** L_1 { *L*. *type* = L_1. *type*; *addType* (**id**. *entry*, L_1. *type*); }

$L \rightarrow$: T { *L*. *type* = *T*. *type*; }

$T \rightarrow$ **integer** { *T*. *type* = *integer*; }

$T \rightarrow$ **char** { *T*. *type* = *char*; }

可以说，在 Pascal 声明中，类型信息是从右向左流动的。本小节开始给出的文法是按从左向右归约设计的，因此不可能在分析期间完成把类型信息填入符号表的计算。而修改后的文法是按从右向左归约设计的，类型信息流向和归约方向的一致性使得属性计算可以在分析期间完成。

4.4 *L* 属性的自下而上计算

本节提出在自下而上分析的框架中实现 *L* 属性定义的方法。它能实现任何基于 LL(1) 文法的 *L* 属性定义，也能实现许多（但不是所有的）基于 LR(1) 的 *L* 属性定义。该方法是对 4.2.3 节自下而上翻译技术的推广。本节所讲的一些技术已经应用于 Yacc 或类似的生成器中。

4.4.1 删除翻译方案中嵌入的动作

4.2.3节的自下而上翻译方法适用于S属性定义，S属性定义的语义动作都可以放在产生式的右端，在归约时执行。虽然L属性定义的继承属性计算需要嵌在产生式右部的不同地方，但是可以通过修改文法而把翻译方案中所有的嵌入动作都变换成只出现在产生式的右端，当嵌入动作只涉及虚拟属性时，这种变换尤其容易。

这种变换在文法中加入推出空串的**标记非终结符**，让每个嵌入动作由不同标记非终结符M代表，并把该动作放在产生式$M\rightarrow\varepsilon$的右端。例如，使用标记非终结符M和N，(4.1)翻译方案

$E\rightarrow TR$

$R\rightarrow +T\ \{print\ ('+');\}R_1 \mid -T\ \{print\ ('-');\}R_1 \mid \varepsilon$

$T\rightarrow$ **num** $\{print\ (\textbf{num}.val);\}$

变换成

$E\rightarrow TR$

$R\rightarrow +TMR_1 \mid -TNR_1 \mid \varepsilon$

$T\rightarrow$ **num** $\{print\ (\textbf{num}.val);\}$

$M\rightarrow\varepsilon\ \{print\ ('+');\}$

$N\rightarrow\varepsilon\ \{print\ ('-');\}$

这两个翻译方案中的文法接受同样的语言，并且对任何输入，它们执行的语义动作是一样的。变换后的翻译方案中，动作都在产生式的右边，所以它们可以在自下而上分析过程中归约产生式右部的时候完成。

4.4.2 分析栈上的继承属性

对产生式$A\rightarrow XY$，自下而上分析器从它的栈顶移开Y和X，并用A代替它们，完成产生式右部的归约。如果X有综合属性$X.s$，4.2.3节的实现把该属性放入栈中与X对应的地方。

因为在Y以下的任何子树归约前，$X.s$的值已经在栈中，所以它的值可以被Y继承。如果Y的继承属性由复写规则$Y.i=X.s$定义，那么在需要使用$Y.i$的地方，可以取$X.s$的值来代替。如果能静态地确定$X.s$在栈中的位置，那么这个想法很容易实现。

例4.12 考虑由表4.6的语法制导定义改写的翻译方案：

$D\rightarrow T$ $\{L.in=T.type;\}$

L

$T\rightarrow$ **int** $\{T.type=integer;\}$

$T\rightarrow$ **real** $\{T.type=real;\}$

$L\rightarrow$ $\{L_1.in=L.in;\}$

L_1 , **id** { *addType* (**id**. *entry*, *L*. *in*) ; }

$L\rightarrow$ **id** { *addType* (**id**. *entry*, *L*. *in*) ; }

该翻译方案使用继承属性,并且继承属性由复写规则传递。下面以输入 int p,q,r 为例,考察在需要 *L*. *in* 值的地方,能否静态地确定 *T*. *type* 在栈中的位置。

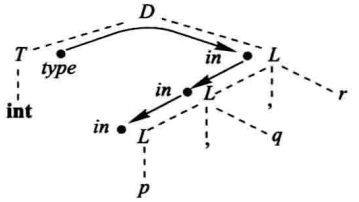


图 4.13 在每个 *L* 结点上,*L*. *in* = *T*. *type*

先用图 4.13 给出该输入的分析树及 *type* 和 *in* 属性之间的依赖关系,以增强直观性。表 4.8 给出分析器面临该输入时的动作序列。为清楚起见,仍然用对应的文法符号代替状态,用实际的标识符代替记号 **id**。在表 4.8 中,每次归约 *L* 产生式右部时,*T* 在栈中刚好处于该右部的下面,因此 *T*. *type* 在栈中的位置可以静态地确定。这个事实可用来访问属性 *T*. *type*。

表 4.8 分析器面临 int p,q,r 时的动作序列

状态	输入	所用产生式
–	**int** p,q,r	
int	p,q,r	
T	p,q,r	*T* →**int**
*T*p	,q,r	
TL	,q,r	*L* →**id**
TL,	q,r	
TL,q	,r	
TL	,r	*L* →*L*,**id**
TL,	r	
TL,r		
TL		*L* →*L*,**id**
D		*D*→*TL*

按照 4.2.3 节的假设,分析栈由 *state* 和 *val* 两部分来实现,并且如果 *stack*[*i*]. *state* 对应文法符号 *X*,那么 *stack*[*i*]. *val* 保存 *X* 的综合属性 *s*。表 4.9 的实现利用了属性 *T*. *type* 在栈中的位置已知这个事实。

表 4.9 *T.type* 的值可用来代替 *L.in*

产生式	代码段
$D \rightarrow TL$	
$T \rightarrow$ **int**	*stack*[*top*].*val*=*integer*;
$T \rightarrow$ **real**	*stack*[*top*].*val*=*real*;
$L \rightarrow L_1$, **id**	*addType*(*stack*[*top*].*val*,*stack*[*top*−3].*val*);
$L \rightarrow$ **id**	*addType*(*stack*[*top*].*val*,*stack*[*top*−1].*val*);

当使用产生式 $L \rightarrow$ **id** 时，**id**.*entry* 在 *val* 栈顶，*T.type* 刚好在它的下面。所以 *addType*(*stack*[*top*].*val*,*stack*[*top*−1].*val*)等价于 *addType*(**id**.*entry*,*L.in*)。同样，因为产生式 $L \rightarrow L$,**id** 右部有三个符号，所以归约时 *T.type* 出现在 *stack*[*top*−3].*val*。其他和 *L.in* 有关的动作是复写规则，它们完全可以省略掉。 □

如果属性的位置不能预测，可以尝试增加标记非终结符，使属性的位置变得可以静态确定。下面通过一个例子来说明。

例 4.13 考虑下面的语法制导定义：

产生式	语义规则	
$S \rightarrow aAC$	$C.i=A.s$	
$S \rightarrow bABC$	$C.i=A.s$	(4.2)
$C \rightarrow c$	$C.s=g(C.i)$	

通过复写规则，*C.i* 继承综合属性 *A.s*。由于在栈中，*B* 可能在、也可能不在 *A* 和 *C* 之间，因此在用产生式 $C \rightarrow c$ 归约时，*C.i* 的值(也就是 *A.s* 的值)在 *stack*[*top*−2].*val* 或者在 *stack*[*top*−1].*val*。

增加一个标记非终结符 *M*，插在(4.2)的第二个产生式右部的 *C* 之前，并增加了属性复写规则。修改后的语法制导定义如下：

产生式	语义规则
$S \rightarrow aAC$	$C.i=A.s$
$S \rightarrow bABMC$	$M.i=A.s$ $C.i=M.s$
$C \rightarrow c$	$C.s=g(C.i)$
$M \rightarrow \varepsilon$	$M.s=M.i$

产生式 $S \rightarrow bABMC$ 和 $M \rightarrow \varepsilon$ 的语义规则保证 $C.i=M.s=M.i=A.s$，也就是对于第二个产生式 $S \rightarrow bABMC$，*C.i* 间接地通过 *M.i* 和 *M.s* 继承 *A.s* 的值。这样，不管在 *aAC* 还是 *bABMC* 的情况下，当使用 $C \rightarrow c$ 时，*C.i* 的值都可从 *stack*[*top*−1].*val* 找到。在使用 $M \rightarrow \varepsilon$ 时，*M.i* 的值可以在 *stack*[*top*−1].*val* 找到。 □

4.4.3 模拟继承属性的计算

如果继承属性并不直接等于某个综合属性，而是它的一个函数，是否能够利用上一小节的办法呢？回答是可以的。在这种情况下，可以用标记非终结符来模拟继承属性的计算。例如，考虑

产生式	语义规则
$S \rightarrow aAC$	$C.i=f(A.s)$
$C \rightarrow c$	$C.s=g(C.i)$

这里，定义 $C.i$ 的规则不是简单的复写规则。如果不完成这个计算，则在栈中取不到 $C.i$ 的值。解决办法是增加一个标记非终结符，把 $f(A.s)$ 的计算移到对标记非终结符归约时进行，如下所示：

产生式	语义规则
$S \rightarrow aANC$	$N.i=A.s \quad C.i=N.s$
$N \rightarrow \varepsilon$	$N.s=f(N.i)$
$C \rightarrow c$	$C.s=g(C.i)$

当按 $N \rightarrow \varepsilon$ 归约时，$N.i$ 的值可以在放 $A.s$ 的地方（$stack[top].val$）找到。当按 $C \rightarrow c$ 归约时，$C.i$ 的值可以在对应于 N 的地方（$stack[top-1].val$）找到。

例 4.14 对于表 4.7 的排版语言例子，可以增加三个标记非终结符 L，M 和 N，以保证当 B 的子树被归约时，所需的继承属性 $B.ps$ 的值出现在分析栈中已知的位置。修改后的语法制导定义见表 4.10。

标记非终结符 L 的作用是完成 $S \rightarrow LB$ 中的属性 $B.ps$ 的初始化。$L \rightarrow \varepsilon$ 的语义规则 $L.s=10$ 使得 $B.ps$ 的初值 10 在栈中有存放的地方，以便后面引用。

标记非终结符 M 和 N（还有 L）的一个共同作用是，保证了在任何情况下向 B 归约时，$B.ps$ 的值总是正好在右部的下面。这个结论的证明留作一个练习。

N 的另一个作用是模拟继承属性的计算。在表 4.7 中，产生式 $B \rightarrow B_1$ **sub** B_2 的语义规则 $B_2.ps=shrink(B.ps)$ 表明 $B_2.ps$ 不是简单地由复写规则定义的。产生式 $N \rightarrow \varepsilon$ 的动作完成这个计算，并将计算结果保留在栈上。

表 4.10 由复写规则设置所有继承属性

产生式	语义规则
$S \rightarrow LB$	$B.ps=L.s \quad S.ht=B.ht$
$L \rightarrow \varepsilon$	$L.s=10$
$B \rightarrow B_1MB_2$	$B_1.ps=B.ps \quad M.i=B.ps \quad B_2.ps=M.s \quad B.ht=max(B_1.ht,B_2.ht)$
$M \rightarrow \varepsilon$	$M.s=M.i$

续表

产生式	语义规则
$B \rightarrow B_1$ **sub** NB_2	$B_1.ps = B.ps$ $N.i = B.ps$ $B_2.ps = N.s$ $B.ht = disp(B_1.ht, B_2.ht)$
$N \rightarrow \varepsilon$	$N.s = shrink(N.i)$
$B \rightarrow$ **text**	$B.ht =$ **text**$.h \times B.ps$

实现表4.10语法制导定义的代码段列在表4.11。在表4.10中，所有的继承属性都是由复写规则来赋值，因而只要知道它们在栈中的位置，就可以获得它们的值。 □

表4.11 表4.10的语法制导定义的实现

产生式	语义规则
$S \rightarrow LB$	$stack[top-1].val = stack[top].val$;
$L \rightarrow \varepsilon$	$stack[top+1].val = 10$;
$B \rightarrow B_1 MB_2$	$stack[top-2].val = max(stack[top-2].val, stack[top].val)$;
$M \rightarrow \varepsilon$	$stack[top+1].val = stack[top-1].val$;
$B \rightarrow B_1$ **sub** NB_2	$stack[top-3].val = disp(stack[top-3].val, stack[top].val)$;
$N \rightarrow \varepsilon$	$stack[top+1].val = shrink(stack[top-2].val)$;
$B \rightarrow$ **text**	$stack[top].val = stack[top].val \times stack[top-1].val$;

可以证明，对于基于LL(1)文法的L属性定义，系统地引入标记非终结符后，可以在LR分析期间完成属性计算。因为每个标记非终结符至多只有一个产生式，因此标记非终结符加入后文法仍然是LL(1)的。任何LL(1)文法一定是LR(1)文法，因此加标记非终结符后不会引起LR分析动作的冲突。

但是对于基于LR(1)文法的L属性定义，情况就不这么简单，标记非终结符的加入有可能使得文法不再是LR(1)的了。

例4.15 文法$L \rightarrow Lb \mid a$是LR(1)的，加了标记非终结符M后的文法

$L \rightarrow MLb \mid a$

$M \rightarrow \varepsilon$

不是LR(1)的。

该语言的句子是$abb\cdots b$的形式。根据这个文法，空归约的个数和b的个数一样多。在面临a时，后面有多少个b是不知道的，因此一定有移进-归约冲突。所以这个加标记非终结符的文法不是LR(1)的。 □

除4.1节所讨论的分析树方法以外，本章具体讨论的都是一边分析一边进行属性计算的方

法。它们的优点是效率较高，缺点是访问结点的次序受到分析方法的限制，因而只能完成 L 属性计算。

对于非 L 属性定义，可以把分析和属性计算分开，先建立分析树（也可以用语法树），然后遍历分析树来计算属性。可以把4.3节的预测分析技术进行推广，仍然为每个非终结符构造一个函数，但是这些函数不含语法分析部分，它们是通过遍历分析树来完成属性计算的递归函数。每个函数对分析树上相应结点的子结点的访问次序由所用产生式的语义规则决定，但不一定是从左到右的。

这种方法不受分析方法的限制，属性计算的次序可在构造编译器时通过分析语法制导定义来得出，而且它可以推广到对分析树需要进行多遍扫描的翻译。

习 题 4

4.1 根据表4.1的语法制导定义，为输入表达式5*(4*3+2)构造注释分析树。

4.2 构造表达式((a*b)+(c))的分析树和语法树：

(a) 根据表4.3的语法制导定义。

(b) 根据图4.9的翻译方案。

4.3 为文法

$S \rightarrow (L) \mid a$

$L \rightarrow L,S \mid S$

(a) 写一个语法制导定义，它输出括号的对数。

(b) 写一个语法制导定义，它输出括号嵌套的最大深度。

4.4 下面的文法定义语言 $L=\{a^n b^n c^m \mid m,n \geqslant 1\}$。写一个语法制导定义，其语义规则的作用是：对不属于语言 L 的子集 $L_1=\{a^n b^n c^n \mid n \geqslant 1\}$ 的句子，打印出错信息。

$S \rightarrow DC$

$D \rightarrow aDb \mid ab$

$C \rightarrow Cc \mid c$

4.5 为下面文法写一个语法制导的定义，它完成一个句子的 **while-do** 最大嵌套层次的计算并输出这个计算结果。

$S \rightarrow E$

$E \rightarrow$ **while** E **do** $E \mid$ **id** $:=E \mid E+E \mid$ **id** $\mid (E)$

4.6 下列文法产生由+作用于整常数或实常数的表达式。两个整数相加时，结果是整型，否则是实型。

$E \rightarrow E+T \mid T$

$T \rightarrow$ **num.num** $\mid$ **num**

(a) 给出决定每个子表达式类型的语法制导定义。